Post-Processing for True Random Bits Generator

Master of Technology
Project-II Report

by

Tirumala Reddy B H

Roll No. EE23MT022

Supervisor:

Prof. Naveen Kadayinti

Prof Gopal Krishna Kamath M

Department of Electrical, Electronics and Communication Engineering

INDIAN INSTITUTE OF TECHNOLOGY DHARWAD

November 14, 2025

Abstract

Cryptographically secure random numbers are the foundation of contemporary cryptographic technologies and secure communication. True Random Number Generators (TRNGs) extract their unpredictability from physical sources of entropy like electrical noise, quantum effects, and other deterministic processes, providing better guarantees than pseudorandom number generators. This research addresses the improvement of TRNG outputs by sophisticated post-processing methods to satisfy rigorous cryptographic applications.

The fundamental problem of TRNG design is to convert physical entropy sources into statistically secure random sequences while removing biases and correlations. Here, we thoroughly assess post-processing techniques such as XOR operations, Von Neumann correction, and cryptographic primitives (DES, PRESENT, and AES), focusing on our new barrel Shifter with the XOR technique. Our implementations show that this hybrid method effectively minimises autocorrelation without sacrificing entropy, with the entire 15 NIST statistical tests for randomness being fully satisfied.

The outcomes prove that correctly designed post-processing can reshape raw TRNG outputs into cryptographically secure sequences without sacrificing throughput or implementation feasibility. This contribution provides practical guidelines for TRNG designers in balancing the vital trade-offs between randomness quality, computational overhead, and security assurancee.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 What is a Random Number?

Each value in a defined set has a chance to be selected with an equal probability, with no discernible pattern or order. In other words, random numbers are unpredictable and independent, meaning one cannot guess the following number from the previous ones. Each number has a specific probability (often equal) of occurring.
For example, whenever we experiment with rolling the dice or tossing the coin, the appearance of numbers or coin flipping will be random and equally distributed.

1.1.1 Types of Random Number Generators

To generate those random numbers, we use random bits/number generators. Those generators are mainly divided into two types:

- True Random Bits/Number Generators(TRNG)
- Pseudorandom Random Bits/Number Generators(TRNG)

Those are generated from physical phenomena (e.g., atmospheric noise, radioactive decay, thermal noise) called True Random Bits/Number Generators(TRNG) because they come from natural sources and are implemented in hardware; due to this, they are Non-deterministic, Unpredictable, and are often used in cryptographic applications.

The second one generates the random numbers using algorithms and an initial value called a seed, called Pseudorandom Bits/Number Generators(PRNG). Although they appear random, they're deterministic and will produce the same sequence if started with the

same seed. The source of generating numbers is mathematical algorithms and formulas. Due to this, they are faster compared to TRNGs

1.1.2 Why do we need them?

Random number generators are used in a wide range of applications, from cryptography to gaming, and the type of generator chosen depends on the task's specific needs, whether it is speed, reproducibility, or security.

In cryptography, we require intense, highly unpredictable, and resistant numbers to attacks. Hence, we require True random number generators (TRNGs) because the numbers must be highly unpredictable and resistant to attacks. These generate encryption keys, passwords, secure tokens, and other security-sensitive values. Similarly, a TRNG ensures the rest remains secure for password and key generation even if part of the sequence is revealed.

In contrast, games and simulations typically use pseudo-random number generators (PRNG) because they are predictable, making them ideal for cases where reproducibility is essential, such as debugging or running repeatable experiments in scientific simulations like Monte Carlo methods. PRNGs are also commonly used in machine learning to initialise neural networks, shuffle data, and split datasets randomly, tasks that benefit from speed and repeatability.

Other applications like hardware-based security modules, lottery systems, statistical sampling, software testing, and soon random generators are used. PRNGS take less time to implement than TRNGs, but for PRNGS, the seed value or number is generated from the TRNG; TRNGs give more value to the random number generators and are used for many more applications.

Due to many user cases of TRNGs, this report contains the building blocks of the TRNG Survey of TRNG Designs, New Methods of Post Processing and Testing of the RNGs.

1.2 Building Blocks of TRNGs

The implementation of TRNG involves capturing unpredictable physical phenomena and converting that natural entropy into digital random numbers. TRNGs are hardware-based, unlike algorithmic PRNGs. Due to this, the TRNGS have three main building blocks: an entropy source, an digitization, and post-processing, as shown in the figure.

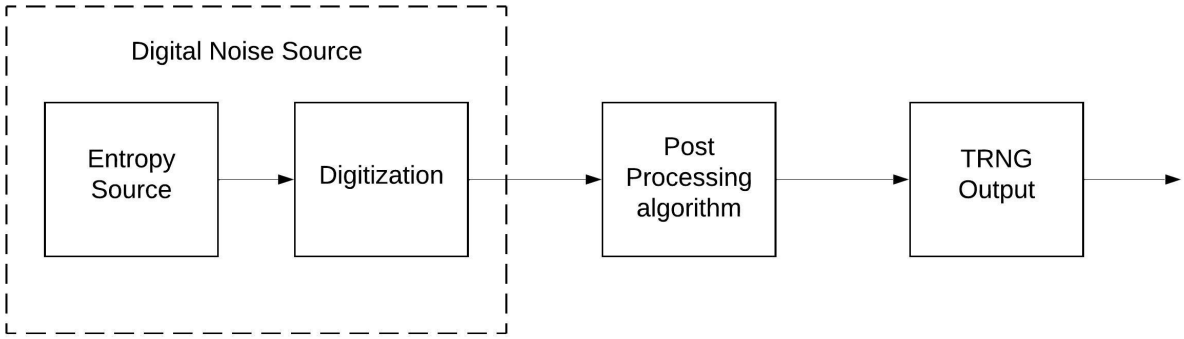


Figure 1.1: Building Blocks of TRNGs

- **Entropy Source:** This block represents the physical source of randomness, such as Thermal noise, Avalanche noise in a diode, Jitter between two clocks
- **Digitization:** The analogue noise is converted into digital bits using an Analog-to-Digital Converter (ADC) or comparator
- **Post-Processing:** Raw random bits may contain biases or correlations, so this block applies mathematical or cryptographic functions for further whitening.

Detailed explanation of each block done in this report of upcoming chapters

Chapter 2

Entropy Source and Digitization

2.1 Entropy Source-1st Block

A True Random Number Generator (TRNG) design usually features three key components. The first block is fundamental, containing the entropy source that produces the raw, unpredictable randomness. The overall quality and reliability of the TRNG hinge on how well this entropy source performs.

The entropy source generates pure randomness by the source of unpredictable physical phenomena. These physical phenomena are noises in circuits like thermal noise, jitter noise, and atom particle movement from devices. The effectiveness of the entropy source plays a crucial role in ensuring the randomness and reliability of the TRNG.

We must harvest randomness from the entropy source without losing its source. The aim is to gather as much entropy as possible from the circuit designs. Different designs are being developed daily to extract the maximum entropy source from the circuit. Some of them are explained in the Survey of TRNG Designs section.

2.2 Digitization-2nd Block

The 2nd block of the TRNG is Digitization, which converts continuous, unpredictable analogue signals into discrete digital bits (0s and 1s), making a bridge between the physical source (analogue) and the digital (binary) that a computer or embedded system can further process. For some TRNGs, the block is integrated into the entropy score block. Hence, there is no need for a design ADC to convert them. This block has some challenges: Voltage thresholds must not drift with temperature, and the output is only taken from

the LSB bits. Making the circuit slow and a waste of entropy. Ensures that we do not lose the entropy while designing this block.

2.3 Survey of TRNG Designs

Efficiency is crucial, with TRNGs requiring compact designs that maximise throughput per area and energy spent. They must use widely available, cost-effective silicon processes, preferably with digital design techniques. It simplifies manufacturing by ensuring seamless microprocessor integration and compatibility with reconfigurable platforms like FPGAs and CPLDs. Here are some of the Surveys on TRNG design implementation methods:

Amplify noise-based TRNGs

Basic noise-based True Random Number Generators (TRNGs)[1] use analogue circuits to amplify noise directly. After the noise was amplified, it was sampled and quantised in circuit devices. The classical amplified noise is shown in the figure(2.1), which uses an amplifier to amplify the small voltage fluctuations caused by electrical devices from a register or semiconductor device to produce randomness of a noise source. Then, the amplified noise is compared with a threshold using a competitor to produce digital signals. This digital signal is sampled and processed to generate a random bit sequence or random numbers.

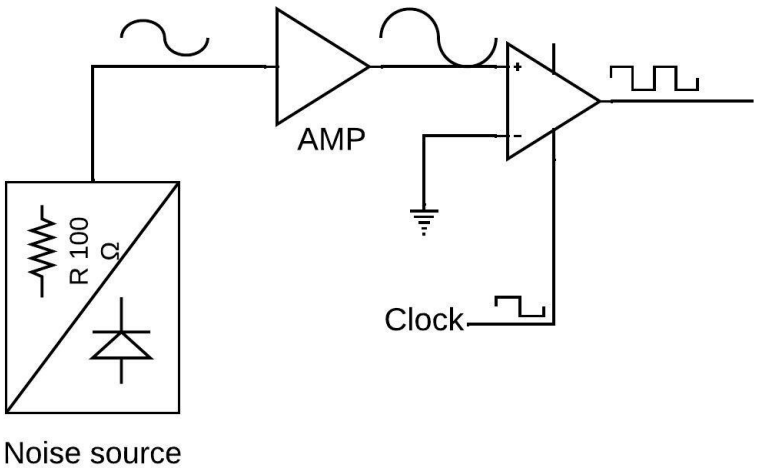


Figure 2.1: Amplify noise-based TRNGs.[1]

To generate randomness using this kind of noise-based TRNG, noises must be amplified to a level where the threshold has no bias, compared by the comparator, which needs

much power to bring the noise level a few orders of magnitude to a digital logic level. Designing such an amplifier is complex.

To overcome this problem, some researchers have come up with solutions. One of the researchers proposed using Si nanodevices[5] to generate high-amplitude device noise, TRNG size was decreased, and high-quality random numbers were obtained. Moreover, to increase the noise in the device using SiN MOSFET[6], the SiN MOSFET is created using a CMOS process with an additional photo mask. Electrons are rapidly injected and ejected between the Si channel and local traps in the SiN layer through direct tunnelling. This process causes a significant conductance change in the transistor, resulting in a substantial drain current fluctuation in the SiN MOSFET, which can produce high noise from the device and reduce the circuit's size complex. Another researcher proposed using a soft breakdown, which does not require an extra photomask like SiN. In this, a large amount of noise is produced due to the large fluctuations in MOS electrical properties after the soft breakdown of MOSFET[7]

However, due to environmental changes and temperature fluctuations, achieving a precise and optimal value in practice cannot be easy. As a result, the digitised bits obtained are often correlated or biased, and further processing is required.

Oscillator-based TRNG

The randomness is generated using the ring oscillator(RO) phase jitter and electrical noise. The ring oscillator, as simple as one inverter, can create the phase jitter[8][9][10]. The RO of an odd number inverts a connected loop or ring, as shown in the figure [2.2]. When the oscillator produces a periodic square wave, it is ideal. However, electrical noise is the real reason why the transition spacing varies. A prior transition's arbitrary uncertainty affects every future transition and will continue indefinitely. This unpredictability will grow as more inverters are added. Due to its ease of usage and straightforward implementation in an FPGA, it is the most popular and commonly used TRNG in digital implementations. However, there are drawbacks and difficulties, such as non-idealities and bias. Overcoming these challenges required post-processing.

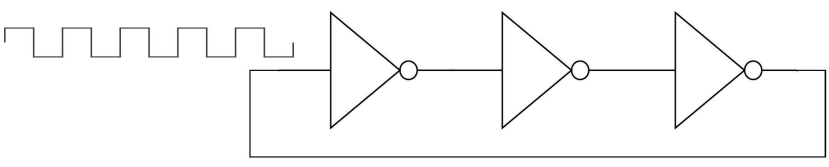


Figure 2.2: The ring oscillator-based TRNG

Meta-stability Based TRNG

As figure [2.2] shows, cross-coupled inverters comprise the meta-stable circuit[11]. When the clock runs low, nodes 'a' and 'b' are charged to high values. When the clock rises to high, nodes settle into meta-stable states. The voltages of nodes 'a and b' can be affected by various disturbances, including thermal, power, and shot noise. Random data can be acquired if the meta-stable condition is broken. During the meta-stable phase, the stable state of nodes 'a' and 'b' can go from high to low or low to high on the noise at nodes 'a' and 'b' as in the figure represented [2.2]. However, the manufacturing process is sensitive to meta-stability and voltage and temperature (PVT) changes. Because of this, Keeping a bi-stable circuit in an unstable condition is difficult[12].

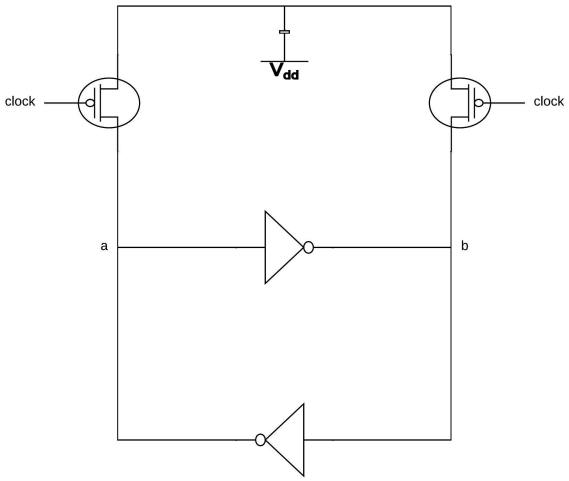


Figure 2.3: Cross-coupled inverter pair: Entropy source

Chaotic TRNGs

Chaos is an ideal entropy source for fast TRNGs because of its large bandwidth, unpredictability, and sensitivity to different events and hardware limits. The chaos-based TRNGs' entropy source is a nonlinear dynamical system operating in a chaotic state. A sampler then collects the entropy, and any errors in the entropy source are hidden by post-processing.

Chaos-based TRNGs[1] have been classified into two types depending on their fundamental behaviour: continuous and discrete time. The future state of continuous-time chaotic systems is identified by differential equations that involve the rate of change corresponding to each factor of the present state.

A continuous-time chaotic system is a time-dependent chaotic system based on a time observation series. The mathematical model of a continuous-time chaotic dynamic system is as follows:

However, analogue circuit blocks, such as op-amps, are required for continuous time chaos-based TRNGs, resulting in huge space and high power consumption.

Where TRNGs based on discrete-time chaos may be produced with fewer components. Differential equations that only apply to the current state generate the future state of discrete-time chaotic systems. Discrete-time chaos-based TRNGs generally require an external clock to drive chaotic dynamics. Thus, the rate of creation and growth of chaotic dynamics depends on the clock frequency. The clock frequency of discrete-time chaos-based TRNGs may be dynamically modified during runtime. This results in lower consumption and higher throughput without requiring any topological changes.

In addition, digital circuits can be used to build a discrete-time chaos-based TRNG. As a result, it is easy to construct a digital integrated circuit, and its circuit has just a few basic electrical components. As a result, it is a viable contender for use in lightweight and hardware-efficient TRNG. In contrast, digital chaotic systems need finite computing accuracy, which may result in pseudo-random output. Thus, creating a continuous random variable that can be employed in digital circuits will be extremely beneficial. This variable can benefit from both the analogue chaotic signal and the digital circuit benefits.

Sigma-Delta ($\Sigma - \Delta$) modulator Based TRNG

The Sigma-Delta ($\Sigma - \Delta$)[13] modulator consists of critical components like a low-resolution ADC (analogue-to-digital converter), a DAC (digital-to-analogue converter) for feedback, and a loop filter that processes the signal and noise, shaping the random output as shown in diagram(2.4. It relies on noise sources such as thermal noise, clock jitter, and comparator noise to generate randomness, which is quantised into a random bit stream through the feedback loop. The core principle involves amplifying and shaping these noise sources to produce a high-entropy random bit stream, making the system a TRNG and an ADC[14][15]. The main advantages of using a $\Sigma - \Delta$ modulator for TRNG include its compact design, leveraging existing components for random number generation and analogue-to-digital conversion, and the robust randomness of multiple noise sources. However, it comes with challenges like analogue circuit complexity, sensitivity to environmental factors (temperature, voltage), and the need for post-processing to remove biases or correlations from the random bits.

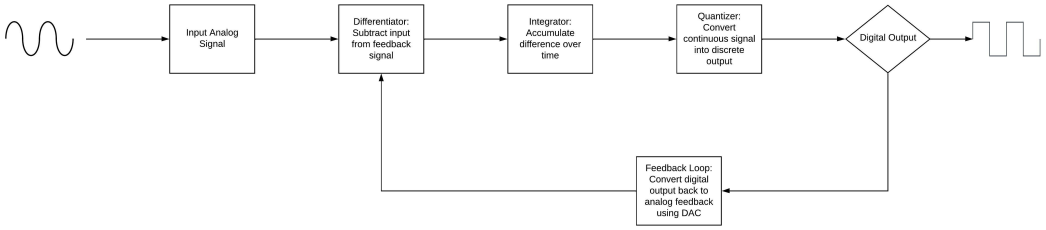


Figure 2.4: Sigma-Delta ($\Sigma - \Delta$) modulator

Chapter 3

Post-Processing

Whatever the raw bits come from the TRNGS, they have some bias and some correlation, which will cause the bits to become predictable. To resolve this problem, the 3rd block will come into the picture, called the Post-Processing. This chapter will look into post-processing, its implementation in TRNGS, and the implementation methods. This project work consisted of a new method of Post-Processing

3.1 Post-Processing-3rd Block

The noise generated from the circuit may not be ideal, so it may introduce bias and correlation, making bits that are not purely random. To improve the randomness, we use post-processing. Post-processing uses mathematically or cryptographically refined data to ensure the bitstream has uniform distribution and independence and meets security and statistical standards.

The post-processing comes after digitising the raw bits from the TRNGs. Where post-processing should be simple and have less power consumption, implementing post-processing as simple as XORing to Von Neumann, correctors can address basic biases. At the same time, more advanced techniques like cryptographic hash functions provide additional security and resilience. There are many different ways of implementing post-processing.

3.2 Survey of Post-Processing Implementation

This section will examine some of the post-processing methods' working principles and efficiency.

3.2.1 XOR Function

XOR post-processing[14][9][10] is among the most popular and easy-to-implement post-processing in digital TRNG circuits. Almost all of the TRNG designers use this type of post-processing. This post-processing reduces the bias of the raw source of TRNG to extract entropy. In the XOR function, the TRNG circuits are connected in parallel. Then, the out fluctuations or bias of one of the TRNGs are averaged, and bits with enhanced unpredictability are obtained, as shown in the table and equation. Although the XOR function gives some benefits, it is highly data-dependent. A decrease in the entropy of one of the TRNG makes the output entirely reliant on the other TRNG. A parallel connection of the TRNGs is needed; due to this, the area and power consumption will be greater using this post-processing type.

$$\epsilon_{\text{intcr}} = 2^{n-1}\epsilon_{\text{raw}}^{n} \tag{3.1}$$

where ϵ_{raw} is the bias of raw random numbers and ϵ_{intcr} ios the bias of internal random numbers.

TRNG 1	TRNG 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Table 3.1: XOR Function/Truth Table

3.2.2 Von Neumann Function

Perfect bias removal may be achieved in ideal circumstances by the use of Von Neumann post-processing. The original random number sequence will be divided into successive but not overlapping 2-bit blocks of data, with the first bits of the other blocks ("01" and "10") being used as the post-processing output and blocks "00" and "11" being discarded, assuming that the bits have the same bias but are independent of one another.As the table shows[3.2]

The benefit of the Von Neumann post-processing approach is that, when the input bits are independent of one another and share the same bias, it carefully removes the bias of

TRNG 1	TRNG 2	Output
0	0	No Data
0	1	0
1	0	1
1	1	No Data

Table 3.2: Von Neumann Function Table

the output bits.
The algorithm only generates an output if the input block is 10 or 01. A lengthy waiting period arises when there is a long sequence of successive "0" or "1" inputs in the input sequence due to the variable output rate of Von Neumann post-processing. The most favourable outcome includes a large amount of entropy waste because the data output rate is only 1/4 of the input rate, and more than 3/4 of the inputs are discarded.

3.2.3 Linear-feedback shift register(LSFR)

A Linear Feedback Shift Register (LFSR)[2] is a shift register whose input bit is a linear function of its previous state. It is commonly used in digital circuits for generating pseudorandom sequences, error detection (CRC), and cryptography. The flip-flops shift the bits left or right with each clock cycle. Selected bits from the register are XORed and fed back into the input. An n-bit LFSR can produce a sequence with a maximum period of 2^n-1. This post-processing method can help to remove bias by spreading entropy and reducing the correlation. Moreover, it can be easily implemented in FPGA/ASIC designs, as shown in the figure(3.1).

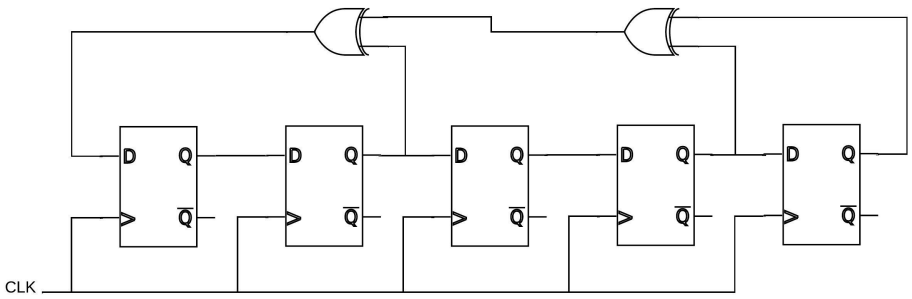


Figure 3.1: Linear-feedback shift register(LSFR)[2]

3.2.4 Overview of the DES Algorithm

The Data Encryption Standard (DES)[3][16] is a symmetric block cipher designed for secure encryption. It is structured as an iterative block cipher, processing 64-bit plaintext blocks using a 56-bit key (with eight parity bits) and producing 64-bit ciphertext blocks.

The algorithm relies on core cryptographic principles of confusion and diffusion, achieved through substitution and permutation operations.

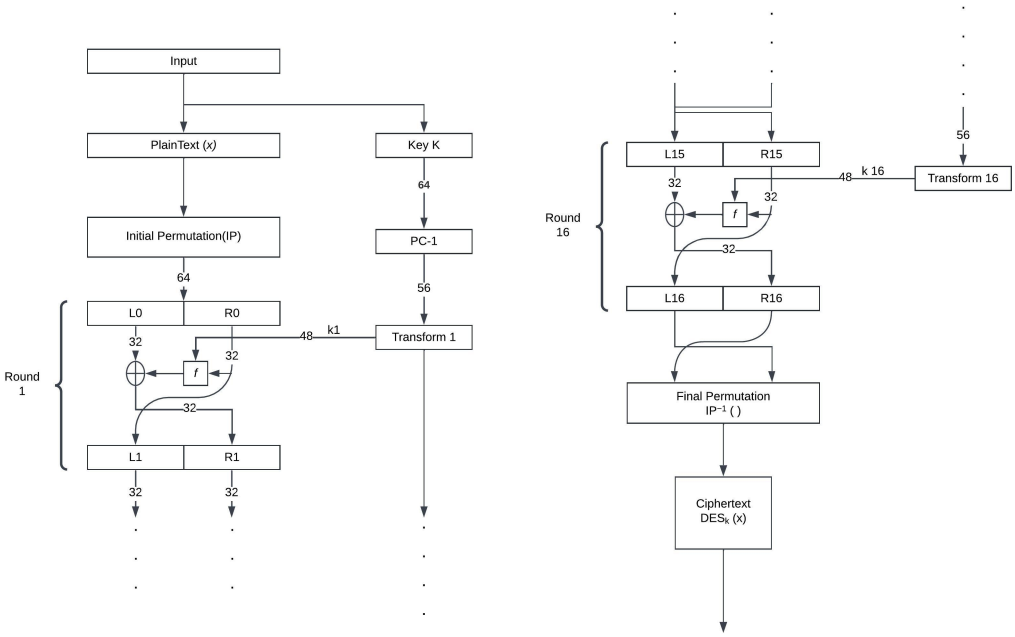


Figure 3.2: DES Algorithm.[3]

Symmetric Cipher and Iterative Process

- **Symmetric Cipher:** DES uses the same key for encryption and decryption.
- **Iterative Algorithm:** Encryption is handled in 16 rounds. Each round performs identical operations but with a different round subkey derived from the main key.
- The iterative design ensures layers of cryptographic transformations, making DES resistant to simple attacks.

Confusion and Diffusion

- **Confusion:** Obscures the relationship between plaintext, key, and ciphertext.
- **Diffusion:** Ensures that small changes in the plaintext or key spread widely across the ciphertext.

DES achieves these principles through:

- Substitution via S-Boxes (for confusion).
- Permutation of bits (for diffusion).

Initial and Final Permutation

- **Initial Permutation (IP):** Before entering the 16 rounds of DES, the plaintext is rearranged through a fixed permutation of its 64 bits. This does not add security but simplifies hardware implementation.
- **Final Permutation (FP):** After the last round, the ciphertext undergoes an inverse permutation, undoing the IP changes.

S-Boxes: Non-Linear Substitution

DES uses 8 S-Boxes[17], fixed substitution tables that operate on six input bits to produce 4 output bits. Each S-Box is designed with cryptographic properties to enhance security.
S-Box Design Properties:

1. **Input to Output Transformation:** Each S-Box transforms six input bits into four output bits using a predefined table.
2. **Non-Linearity:** No single output bit is a simple linear combination of input bits.
3. **Output Variance:** If the lowest and highest bits of the input are fixed and the four middle bits vary, all possible 4-bit output values occur exactly once.
4. **Single-Bit Differences:** If two inputs differ by exactly one bit, their outputs differ in at least 2 bits.
5. **Middle-Bit Differences:** If two inputs differ in their middle bits, their outputs differ in at least 2 bits.
6. **Position-Based Differences:** If two inputs differ in the first two bits but are identical in the last two, their outputs must differ.
7. **Input-Output Difference Restrictions:** For any nonzero 6-bit input difference, 8 out of 32 input-output pairs produce the same output difference.
8. **Collision Probability:** A collision (zero output difference) at the combined 32-bit output of all 8 S-Boxes is possible only for three adjacent S-Boxes.

These properties ensure robustness against linear and differential cryptanalysis.

Key Schedule: Subkey Generation

- DES uses a 56-bit master key to derive 16 round keys k_i for encryption.
- **Key Compression:** The 56-bit key is permuted and compressed using a fixed table, discarding eight parity bits.

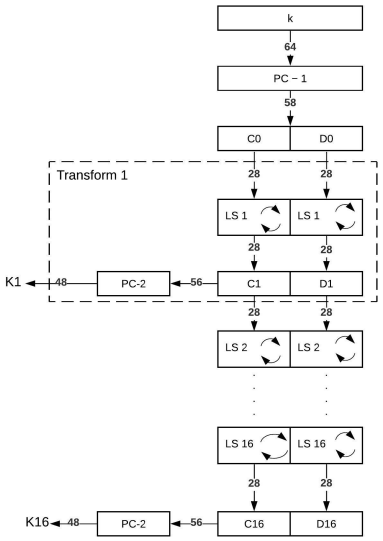


Figure 3.3: DES Key Schedule.[3]

- **Key Rotation:** The compressed key is split into two 28-bit halves. These halves are cyclically left-shifted (rotated) during each round according to a predefined schedule.
- **Key Permutation:** After rotation, another permutation is applied to produce the round key.

Each subkey is unique for its respective round, ensuring that every round introduces new cryptographic strength.

Working Principle

Each DES round applies the following steps as represented in the figure(3.2):

1. **Split:** The 64-bit input block is split into two 32-bit halves: L (left half) and R (right half).
2. **Expansion:** The R half is expanded from 32 bits to 48 bits using a fixed expansion table, allowing it to be XORed with the 48-bit round key.
3. **XOR with Key:** The expanded R is XORed with the round subkey k_i.
4. **Substitution:** The XOR result is divided into eight 6-bit segments, each passed through an S-Box to yield a 4-bit output. These 4-bit outputs combine to form a 32-bit result.
5. **Permutation:** The 32-bit output of the S-Boxes undergoes a fixed permutation to ensure diffusion.

6. **Final XOR:** The permuted result is XORed with the L half, and the R half becomes the new L.

This process repeats for 16 rounds. After the final round, the L and R halves are swapped and combined, followed by the final permutation to produce the ciphertext.

Advantages of DES

While DES is now considered outdated for many applications due to its short key length, it still offers some advantages that contributed to its widespread use in earlier cryptographic systems:

- **Proven Security (Historical):** For its time, DES provided robust security against non-sophisticated attackers, effectively leveraging confusion and diffusion principles.
- **Simplicity of Implementation:** DES relies on simple operations like XOR, substitution, and permutation. This simplicity enabled efficient hardware and software implementations.
- **Standardized Algorithm:** DES became the first widely adopted encryption standard, allowing interoperability across different systems and devices.
- **Efficient Hardware Performance:** The design of DES, particularly its Feistel structure, is well-suited for hardware implementation, providing fast encryption and decryption in constrained environments.
- **Iterative Design for Scalability:** Its 16-round iterative process ensures a balance between performance and security, making it adaptable for varying levels of resource availability.
- **Foundation for Modern Cryptography:** DES laid the groundwork for developing modern cryptographic techniques, such as Triple DES (3DES) and AES, as an essential step in advancing cryptographic research.

Despite these advantages, DES's key length and vulnerability to brute-force attacks have rendered it insufficient for contemporary security needs.

3.2.5 Overview of the PRESENT Algorithm

The PRESENT cipher[3][16][4] is a lightweight block cipher for resource-constrained devices such as RFID tags and IoT devices. It operates on a 64-bit block size with keys of either 80 or 128 bits, making it ideal for low-power and cost-sensitive applications. The algorithm consists of 31 rounds of processing, using simple operations to balance efficiency and security.

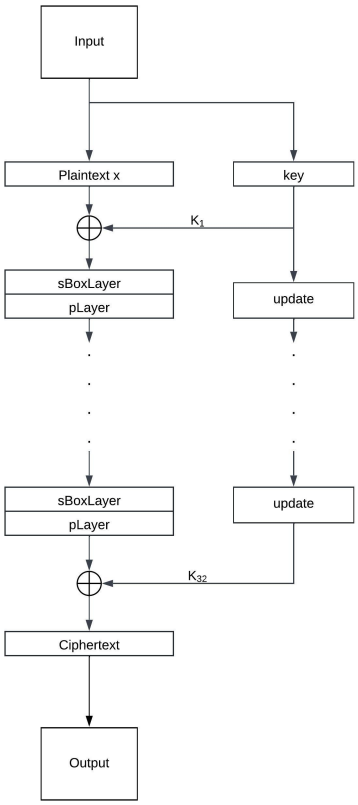


Figure 3.4: PRESENT Algorithm.[3][4]

AddRoundKey

At the beginning of each round, the round key K_i is XORed with the current state of the plaintext.

- This operation integrates the key into the encryption process, ensuring that the ciphertext depends on the key.
- XOR is chosen for its efficiency and reversibility, as $A \oplus B \oplus B = A$.
- The AddRoundKey step makes the cipher resistant to attacks like brute force by introducing key dependency in every round.

sBoxLayer

The substitution layer (*sBoxLayer*) introduces non-linearity to the cipher.

- PRESENT uses a 4-bit S-box that replaces each state nibble (4 bits) with another according to a fixed lookup table.
- This layer ensures that the cipher resists linear and differential cryptanalysis.

- The S-box is carefully designed to satisfy cryptographic properties such as balanced outputs and strong non-linearity.

pLayer

The permutation layer (*pLayer*) provides diffusion, ensuring that small changes in the plaintext or key affect the entire ciphertext.

- In the *pLayer*, bits of the state are shuffled according to a fixed permutation pattern.
- This step breaks any direct relationships between input and output bits, spreading the influence of each bit across the entire block.

Key Schedule

The key schedule generates the round keys K_i from the original master key K.

- The master key is rotated and subjected to bitwise manipulations in each round.
- The first 64 bits of the manipulated key are extracted as the round key.
- For enhanced security, the key schedule applies substitution (using the S-box) and XOR operations with a round counter to ensure that each round key is unique.

Working Principle

The PRESENT cipher operates based on a substitution-permutation network (SPN), which combines confusion and diffusion to secure data. The algorithm consists of the following main operations, as shown in figure(3.4:

- **Substitution (Confusion):** The *sBoxLayer* applies a non-linear substitution to the state, ensuring that the relationship between plaintext, key, and ciphertext is obscured.
- **Permutation (Diffusion):** The *pLayer* shuffles the bits of the state, spreading the influence of each input bit across the output block to eliminate patterns.
- **Key Mixing:** A round key, derived from the master key through a key schedule, is XORed with the state in each round, adding a layer of key dependency.
- **Iterative Processing:** These operations are repeated over 31 rounds, each introducing new cryptographic strength. The high number of rounds compensates for the simplicity of individual operations.
- **5. Key Scheduling:** The round keys are generated by modifying and rotating the master key deterministically, ensuring uniqueness across rounds.

Advantages of PRESENT

- **Lightweight:** PRESENT is optimised for hardware and power-constrained environments, making it suitable for IoT and embedded systems.
- **Simplicity:** Its primary operations, like XOR, substitution, and permutation, allow straightforward implementation.
- **High Security:** Despite its simplicity, the high number of rounds and carefully designed *sBoxLayer* and *pLayer* ensure resistance to common cryptographic attacks.
- **Scalability:** Using either an 80-bit or 128-bit key provides flexibility for varying security needs.

3.2.6 Overview of the AES Algorithm

The Advanced Encryption Standard(AES)[3][16] algorithm comprises the following steps, repeated for each encryption or decryption round:

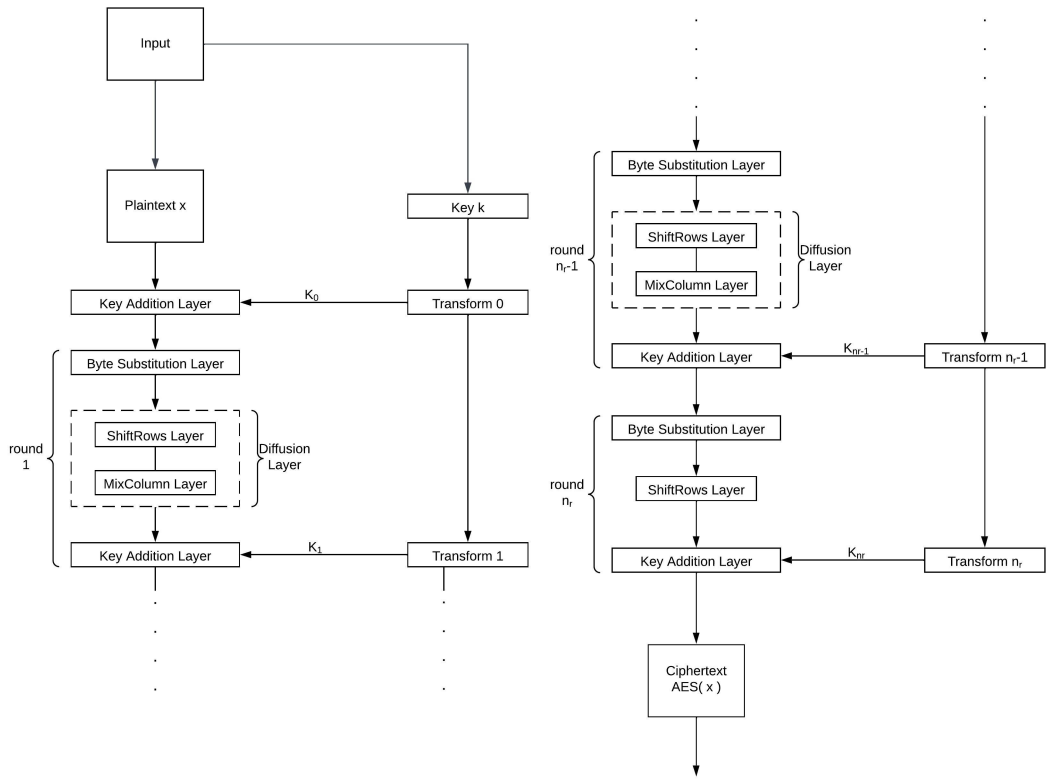


Figure 3.5: AES Algorithm.[3]

Key Addition Layer

At the start of each round, the current state is XORed with a unique round key derived from the main key. This step integrates the key into the encryption process and ensures that the output depends on the key.

Byte Substitution Layer (S-Box)

This layer introduces non-linearity by replacing each byte in the state with a corresponding byte from a precomputed lookup table (S-Box)[18]. The S-Box is constructed using mathematical transformations in the finite field $GF(2^8)$, ensuring resistance against linear and differential cryptanalysis.

Diffusion Layer

The diffusion layer spreads the influence of each input byte across the entire state to obscure patterns:

- **ShiftRows Sublayer**: Rows of the state matrix are cyclically shifted by different offsets to disrupt the alignment of bytes.
- **MixColumns Sublayer**: Data within each column of the state is mixed using matrix multiplication in $GF(2^8)$, ensuring that changes to one byte affect all bytes in a column.

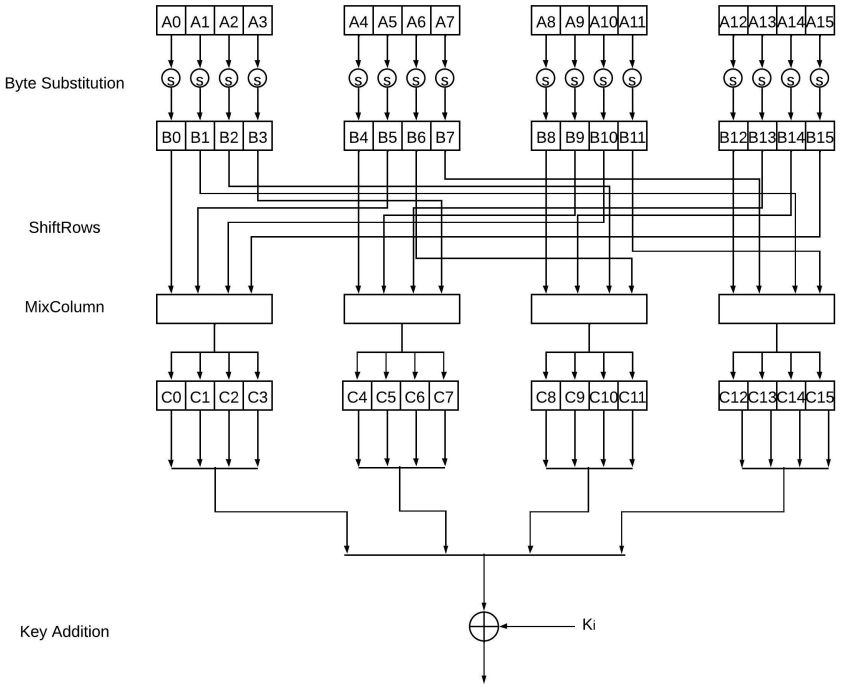


Figure 3.6: AES Diffusion Layer.[3]

Key Schedule

The round keys are generated from the primary key using a key expansion process. This involves transformations such as rotation, substitution using the S-Box, and XOR with a round constant, ensuring each round key is unique and unpredictable.

The final round excludes the MixColumns step, completing the encryption or decryption process.

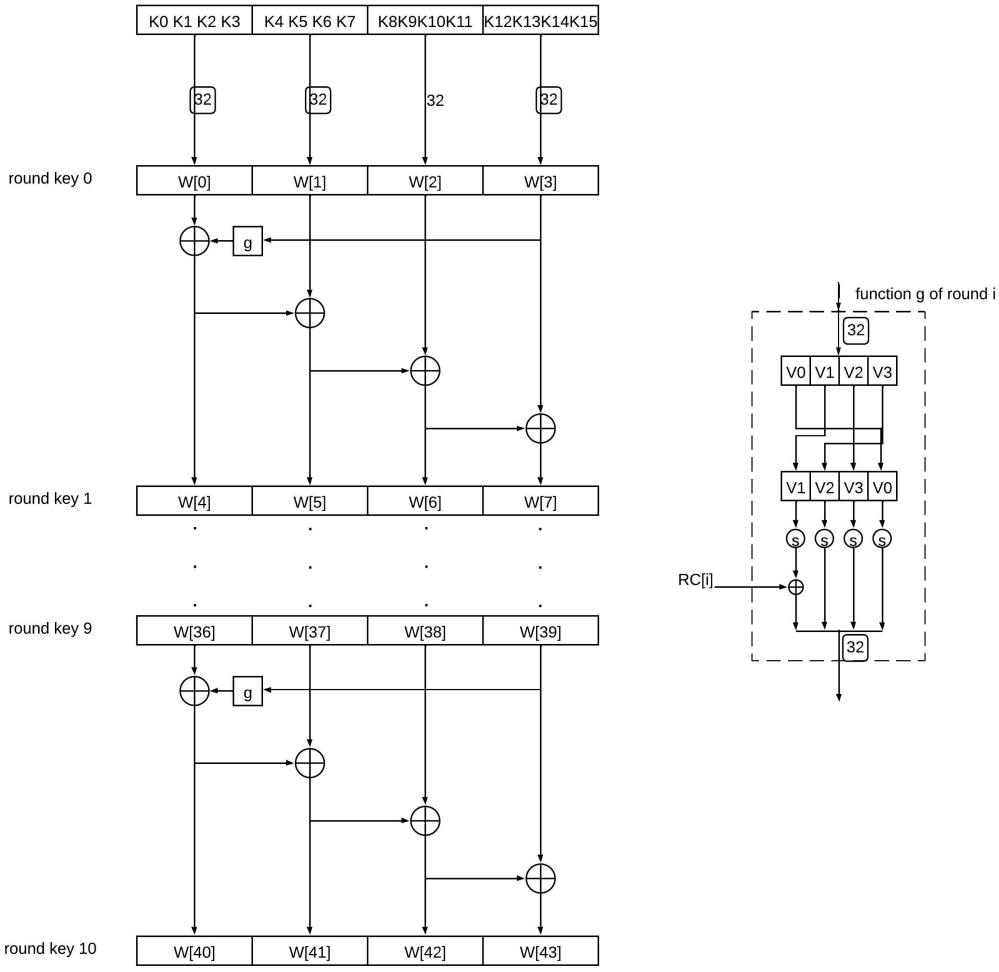


Figure 3.7: AES Key Schedule.[3]

Working of AES Algorithm

The Advanced Encryption Standard (AES) is a symmetric block cipher designed to encrypt and decrypt data securely. It operates on fixed block sizes of 128 bits using key lengths of 128, 192, or 256 bits, making it suitable for various security requirements. AES employs a substitution-permutation network (SPN) to achieve confusion and diffusion,

which are fundamental principles of cryptography, as in the figure (3.5). The algorithm is structured into several rounds, where the number of rounds depends on the key length:

- 10 rounds for a 128-bit key,
- 12 rounds for a 192-bit key, and
- 14 rounds for a 256-bit key.

Each round involves a series of well-defined transformations, including byte substitution, shifting rows, mixing columns, and adding a round key.

Key Features of AES

- **Security**: AES is highly secure due to its combination of substitution, permutation, and key-dependent operations. The algorithm resists known cryptographic attacks, including brute force, differential, and linear cryptanalysis.
- **Efficiency**: AES is designed to perform efficiently in hardware and software implementations. Its straightforward structure allows for fast encryption and decryption, even on resource-constrained devices.
- **Key Length Flexibility**: AES supports key lengths of 128, 192, and 256 bits, allowing users to choose the level of security appropriate for their needs. Longer keys provide greater resistance against brute-force attacks.
- **Scalability**: The modular design of AES makes it scalable for use in various applications, from securing data on embedded devices to protecting classified government information.
- **Standardisation**: As an international standard adopted by NIST, AES is widely used across industries for securing sensitive data and ensuring interoperability between systems.

3.3 New Method of Post-Processing

Using the cryptography algorithm as a post-processing for TRNGs makes it more secure. It reduces the bias and correlation, but utilising this algorithm makes it more complex and challenging to understand. We developed a new post-processing method to overcome this challenge, replacing the S-box with the barrel shifter and key as the shift amount to the barrel shifter. This section discusses the detailed implementation, different approaches, and challenges faced while implementing this method.

3.3.1 Barrel Shifter Only

Overview

The method implements a complex bit-level encryption/transformation algorithm using multiple layers of barrel-shifting operations. The code processes binary data through nested transformations at different bit-length levels (64-bit, 32-bit, 16-bit, 8-bit, and 4-bit). Performs multi-level bit-shifting operations in loops

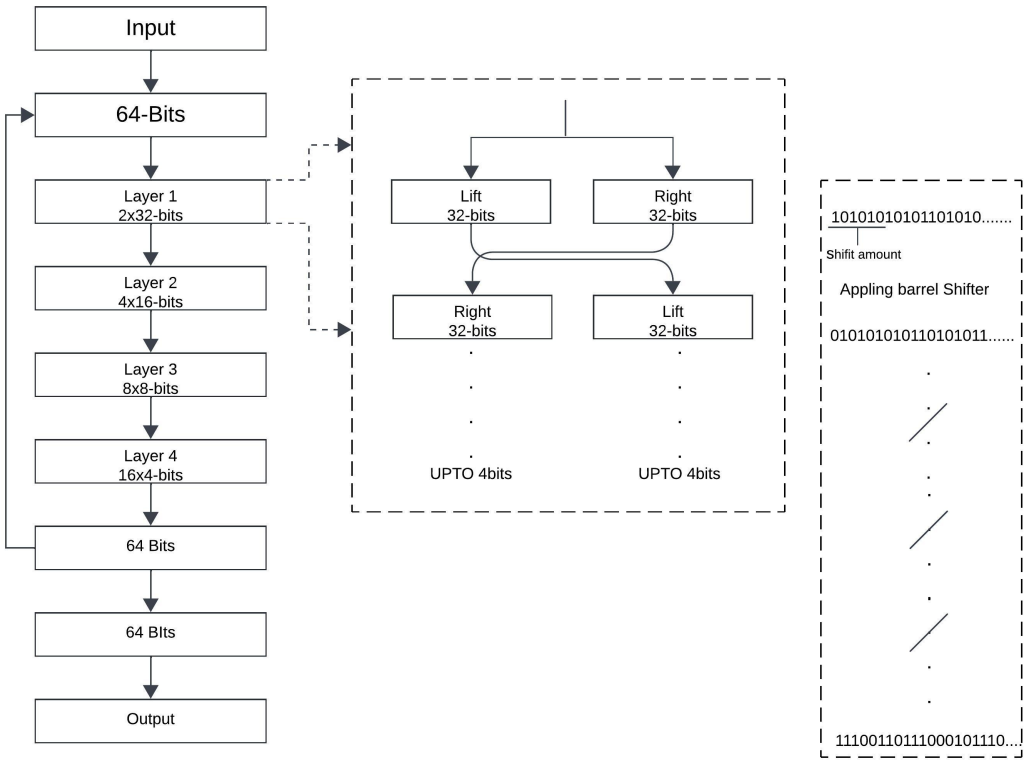


Figure 3.8: Barrel Shifter Only.

Working Principle

- The input is taken from the TRNG's of the output.
- From that process, 64-bit blocks are generated from the input.
- The block processing with splitting, shifting, XORing and recombining resembles Feistel networks, as shown in the figure
- Bit Shifting Hierarchy form 32-bit blocks → 16-bit → 8-bit → 4-bit and then XORing.

Barrel Shifting Operations

The algorithm performs barrel shifting at multiple bit-width levels:

- For 32-bit blocks:
  - Split 64-bit block into left/right 32-bit halves
  - Determine shift amounts from the first 5-6 bits of each half
  - Perform left barrel shifts and swap halves
- For 16-bit blocks:
  - Further, split 32-bit blocks into 16-bit quarters
  - Determine shift amounts from the first 5-8 bits
  - Perform left barrel shifts and complex swapping
- For 8-bit blocks:
  - Split 16-bit blocks into 8-bit chunks
  - Determine shift amounts from the first 3-7 bits
  - Perform left barrel shifts and cross-swapping
- For 4-bit blocks:
  - Split 8-bit blocks into 4-bit nibbles
  - Determine shift amounts from the first 2-3 bits
  - Perform left barrel shifts and random swapping

- After processing all levels, recombine all 4-bit blocks back into 64-bit blocks.
- This process will go through hundreds and thousands of iterations.

Challenges faced

Initially, the shift amount is used by the randi function from the Matlab code, and replacing the randi function with the input bits itself by using autocorrelation reduction takes up more iterations because using the same amount of shifting does not reduce the correlation, instead of using the same amount bits we used the differt amount of bit in each shifting operation.

Observation

The correlation was reduced by using the barrel-shifting operations, but the base was not reduced as expected. Moreover, a larger number of iterations is needed to reduce the correlation. Due to this, the time of execution and power consumption will be higher.

3.3.2 Barrel Shifter & XORing((Final)

Overview

As per previous implementation observations, to reduce the bias and reduce the iterations number, the XORing is done with the input and keeps the remaining operation the same as the previous one, a complex bit-level encryption/transformation algorithm using multiple layers of barrel-shifting operations. The code processes binary data through nested transformations at different bit-length levels (64-bit, 32-bit, 16-bit, 8-bit, and 4-bit). Performs multi-level bit-shifting operations in loops

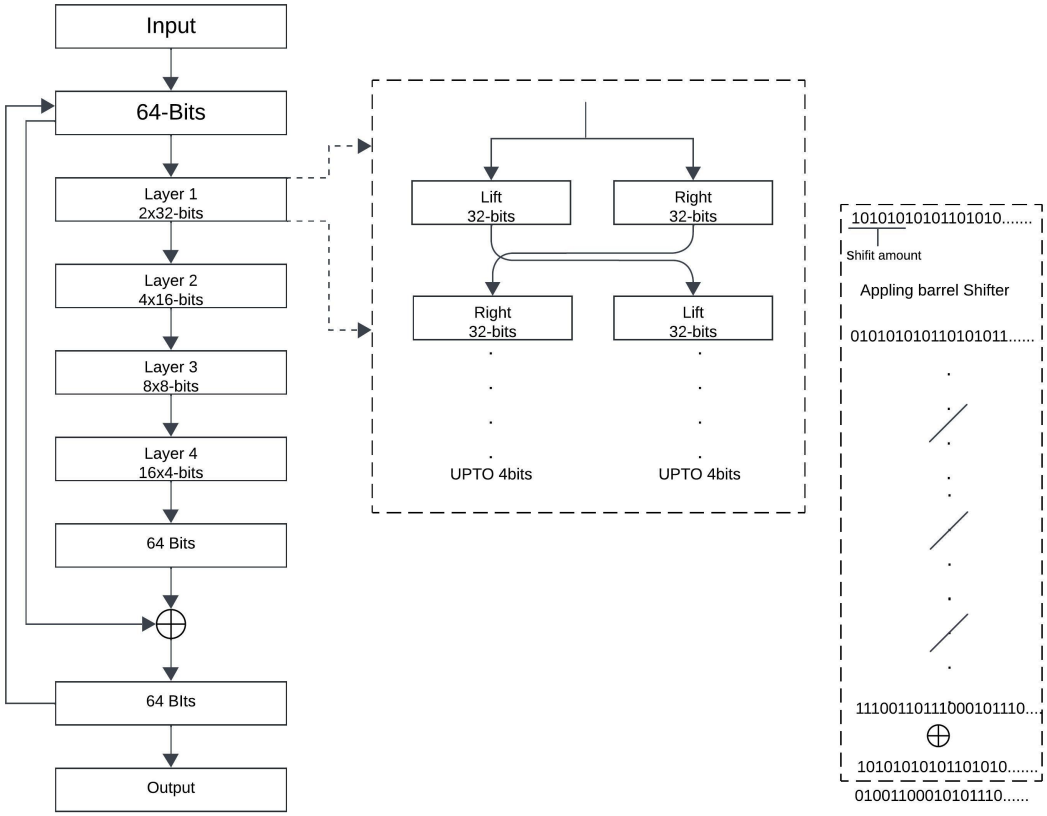


Figure 3.9: Barrel Shifter & XORing((Final)

Working Principle

- The input is taken from the TRNG's of the output.

- From that process, 64-bit blocks are generated from the input.
- The block processing with splitting, shifting, XORing and recombining resembles Feistel networks, as shown in the figure
- Bit Shifting Hierarchy form 32-bit blocks → 16-bit → 8-bit → 4-bit and then XORing.

Barrel Shifting Operations

The algorithm performs barrel shifting at multiple bit-width levels:

  - For 32-bit blocks:
    - Split 64-bit block into left/right 32-bit halves
    - Determine shift amounts from the first 5-6 bits of each half
    - Perform left barrel shifts and swap halves
  - For 16-bit blocks:
    - Further, split 32-bit blocks into 16-bit quarters
    - Determine shift amounts from the first 5-8 bits
    - Perform left barrel shifts and complex swapping
  - For 8-bit blocks:
    - Split 16-bit blocks into 8-bit chunks
    - Determine shift amounts from the first 3-7 bits
    - Perform left barrel shifts and cross-swapping
  - For 4-bit blocks:
    - Split 8-bit blocks into 4-bit nibbles
    - Determine shift amounts from the first 2-3 bits
    - Perform left barrel shifts and random swapping

- After processing all levels, recombine all 4-bit blocks back into 64-bit blocks.
- After recombining all 4-bit blocks back into 64-bit, the XOR operation will be done and fed back to the processing
- This process will go through tens to hundreds of iterations.

Observation

The correlation was reduced by adding XOR, and the bias was reduced as expected due to the non-linear properties of the XOR. However, more iterations are still needed to reduce the correlation and bias. Due to this, the time of execution and power consumption will be higher in hardware implementation.

3.3.3 Barrel Shifter & XOR Operations (Neighbour Blocks)

Overview

As per previous implementation observations, the XOR operation reduced the correlation and bias to some extent, so doing XOR with neighbouring blocks makes more non-linearity in the processing, in which a small change in input makes output change significantly. Moreover, the remaining implementation goes as the previous one, a complex bit-level encryption/transformation algorithm using multiple layers of barrel-shifting operations. The code processes binary data through nested transformations at different bit-length levels (64-bit, 32-bit, 16-bit, 8-bit, and 4-bit). Performs multi-level bit-shifting operations in loops

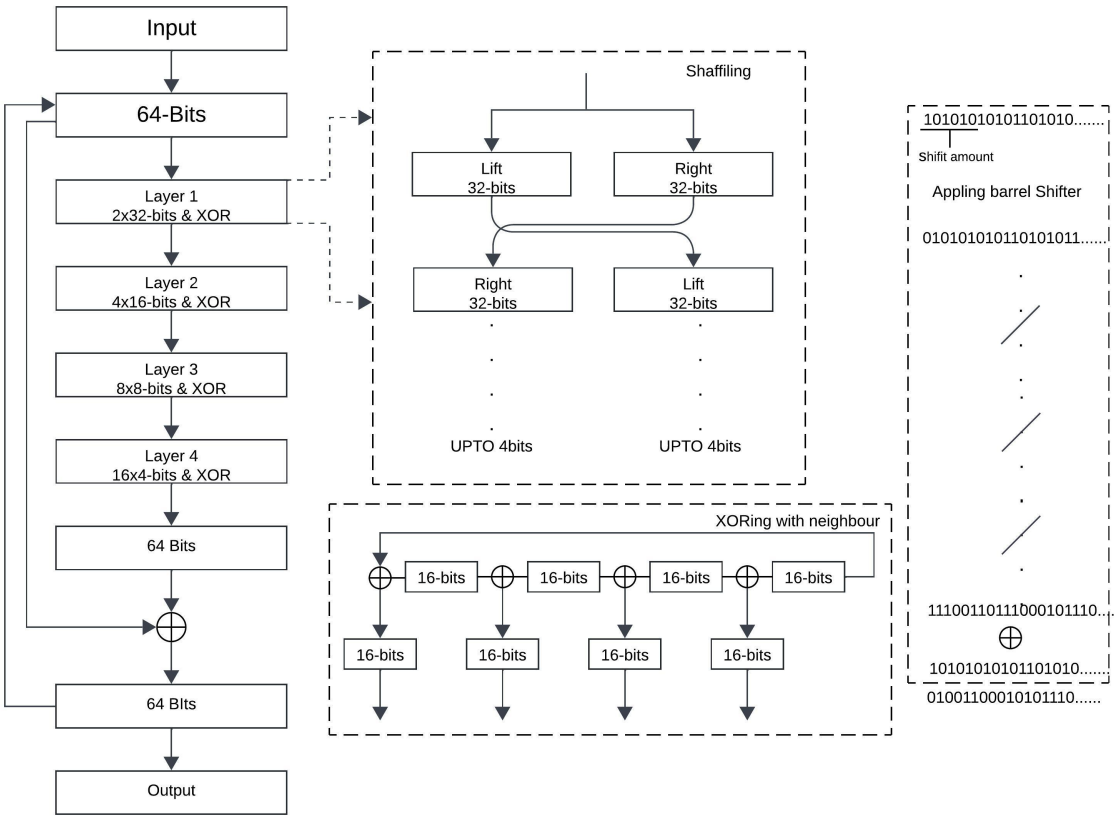


Figure 3.10: Barrel Shifter & XORing(Neighbour Blocks)

Working Principle

- The input is taken from the TRNG's of the output.
- From that process, 64-bit blocks are generated from the input.
- The block processing with splitting, shifting, XORing and recombining resembles Feistel networks, as shown in the figure
- Bit Shifting Hierarchy form 32-bit blocks → 16-bit → 8-bit → 4-bit and then XORing.

Barrel Shifting Operations

The algorithm performs barrel shifting at multiple bit-width levels:

- For 32-bit blocks:
  - Split 64-bit block into left/right 32-bit halves
  - Determine shift amounts from the first 5-6 bits of each half
  - Perform left barrel shifts and swap halves
- For 16-bit blocks:
  - Further, split 32-bit blocks into 16-bit quarters
  - Determine shift amounts from the first 5-8 bits
  - Perform left barrel shifts and complex swapping
  - Now perform the XOR operation with neighbouring blocks
- For 8-bit blocks:
  - Split 16-bit blocks into 8-bit chunks
  - Determine shift amounts from the first 3-7 bits
  - Perform left barrel shifts and cross-swapping
  - Now perform the XOR operation with neighbouring blocks
- For 4-bit blocks:
  - Split 8-bit blocks into 4-bit nibbles
  - Determine shift amounts from the first 2-3 bits
  - Perform left barrel shifts
  - Now perform the XOR operation with neighbouring blocks

- After processing all levels, recombine all 4-bit blocks back into 64-bit blocks.
- After recombining all 4-bit blocks back into 64-bit, the XOR operation will be done and fed back to the processing
- This process will go through tens of iterations.

Observation

It breaks the correlation and reduces the bias of data by XORing due to the non-linear properties of the XOR. However, more iterations are still needed to reduce the correlation and bias. Due to this, the time of execution and power consumption will be higher in hardware implementation.

3.3.4 Impact on Autocorrelation

As for the previous approaches, which did not produce the expected results, we separated them into parts where we experimented with each block separately, like for the 32-bit block only applying Barrel Shifting Operations and for the remaining blocks of 16, 8 and 4-bit. By doing this experiment, the 4-bit block showed more efficiency than the other blocks in reducing the correlation. Due to this, further dividing into 2-bit blocks does the operation but does not reduce the correlation. The Bit Shifting Operations done for the 32-bit blocks → 16-bit → 8-bit in the hierarchy formed by removing the 4-bit blocks did not reduce the correlation as with the 4-bit blocks. So, the 4-bit block is important for processing.

3.3.5 Barrel Shifter & XOR Operations(4-bit Focus)

Overview

The observation from the impact of autocorrelation some changes have and where it initially started from 64-bit → 32-bit blocks → 16-bit → 8-bit → 4-bit now made some modifications where it starts from 64-bit → 4-bit blocks → 8-bit → 16-bit → 32-bit. Moreover, performing the same as complex bit-level encryption/transformation algorithms using multiple layers of barrel-shifting operations. The code processes binary data through nested transformations at different bit-length levels (64-bit, 4-bit, 8-bit, 16-bit, and 32-bit). Performs multi-level bit-shifting operations in loops

Working Principle

- The input is taken from the TRNG's of the output.
- From that process, 64-bit blocks are generated from the input.

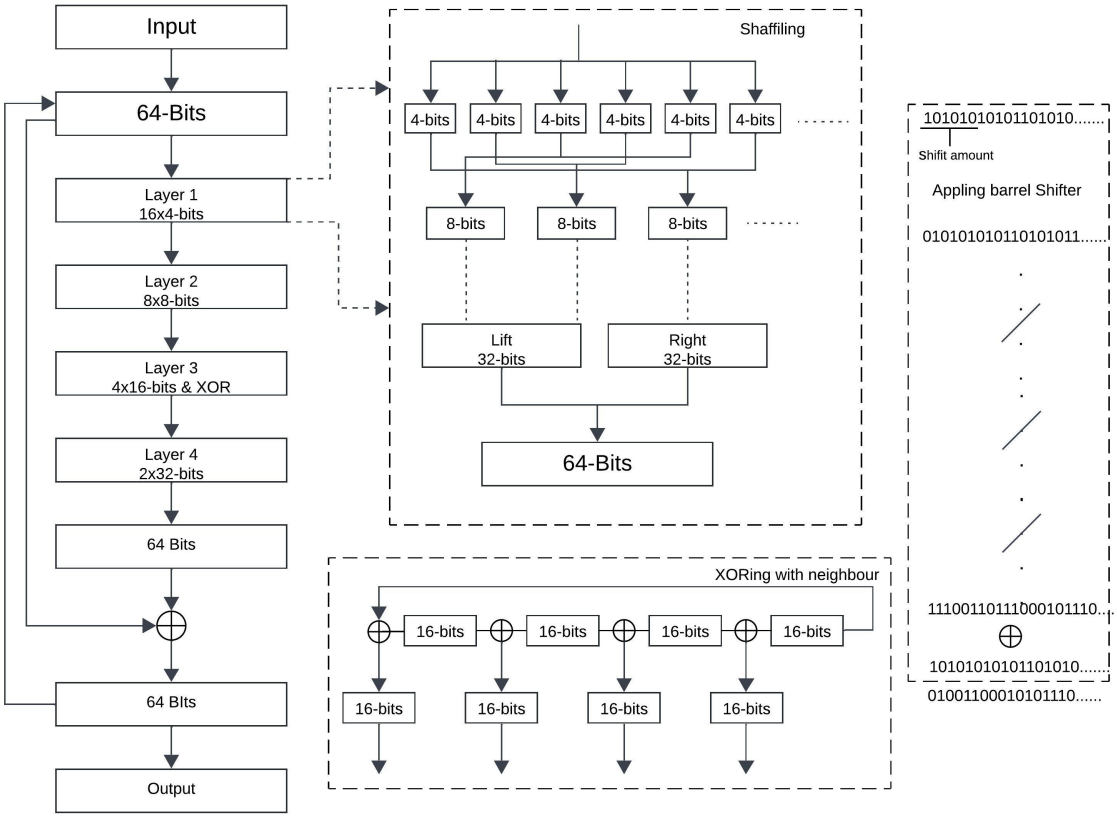


Figure 3.11: Barrel Shifter & XOR Operations(4-bit Focus)

- The block processing with splitting, shifting, XORing and recombining resembles Feistel networks, as shown in the figure
- Bit Shifting Hierarchy form 4-bit blocks → 8-bit → 16-bit → 32-bit
  Barrel Shifting Operations
  The algorithm performs barrel shifting at multiple bit-width levels:
  - For 4-bit blocks:
    - Split 64-bit blocks into 16 4-bit nibbles
    - Determine shift amounts from the first 2-3 bits
    - Perform left barrel shifts and random swapping
    - Combine into 8-bit blocks with two 4-bit blocks
  - For 8-bit blocks:
    - Determine shift amounts from the first 3-7 bits
    - Perform left barrel shifts and cross-swapping
    - Combine 16-bit blocks with two 8-bit blocks.

  - For 16-bit blocks:
    * Determine shift amounts from the first 5-8 bits
    * Perform left barrel shifts and complex swapping
    * Now perform the XOR operation with neighbouring blocks
    * Combine 32-bit blocks with two 16-bit blocks.
  - For 32-bit blocks:
    * Determine shift amounts from the first 5-6 bits of each half
    * Perform left barrel shifts and swap halves
- After processing all levels, recombine 32-bit halves back into 64-bit blocks.
- After recombining blocks back into 64-bit, the XOR operation will be done and fed back to the processing
- This process will go through two to four iterations.

Observation

The correlation and bias were reduced within a few iterations. Compared to other implementations, the XOR operation was done on 16-bit blocks. Because it will rack the continuous ones and zeroes if we do it on 4-bit and 8-bit blocks, where it fails the NIST test. The next chapter, Testing of the Randomness, explains NIST tests and how they are conducted on random bits.

3.3.6 Advantages of These techniques

We went through the same changes in the processing and finally found the best performance, which will start from the 4-bit blocks for the post-processing. Now we will see the advantages of this method/algorithm over other post-processes like XOR, Van Nummen and cryptography algorithms.

- **Data loss:**There is no data loss during this post-processing. It will take 64 bits as input and give processed 64 bits as output.
- **Area:** It requires a lesser area compared to other post-processing, like XOR, which requires two to three TRNG circuits for the post-processing.
- **Implementation:** It is less complex to implement than the cryptography algorithm in the FPGA/circuits.

- **KEY:** It does not require any keys like cryptography from the other source to work effectively. The shift amount will be taken from the input bits for processing.
- **Efficiency:** The bias and correlation of the bits were almost removed

3.4 Comparison

This section will compare proposed post-processing methods with implementation, key features, challenges, and observations

Table 3.3: Comparison of Barrel Shifter Post-Processing Methods

Method	Processing Order	Key Features	Challenges	Observations
Barrel Shifter Only	64→32→16→8→4-bit	• Multi-level barrel shifting • Shift amounts derived from input bits	• High iterations needed • Correlation reduction is insufficient	• Correlation reduced, but bias remains • High power/execution time
Barrel Shifter + XOR (Final)	64→32→16→8→4-bit	• Adds XOR after recombination • Non-linear feedback	• Still requires many iterations	• Better bias reduction • Still high resource usage
Barrel Shifter + XOR (Neighbour Blocks)	64→32→16→8→4-bit	• XOR with neighbouring blocks at each level • Enhanced non-linearity	• Complex swapping logic • Higher hardware overhead	• Stronger correlation/bias reduction • More efficient than previous methods
Barrel Shifter + XOR (4-bit Focus)	64→4→8→16→32-bit	• Starts with 4-bit blocks (most effective) • XOR at 16-bit level	• Optimal shift amount selection critical	• Fastest bias/correlation reduction • Fewer iterations (2-4)

Note: All methods evaluated on 64-bit input

Chapter 4

Testing the random bits

Finally, to conclude, the randomness of the bits generated by the TRNG/PRNG needs to be verified so to verify to test the randomness of the bits. There are some test suites for randomness, like NIST, and some entropy tests, like Shonnen entropy

4.1 Types of Tests

- **NIST Statistical Test Suite (SP 800-22)**
  - Developed by: National Institute of Standards and Technology (NIST)
  - Year: 2001 (updated in 2010)
  - Purpose: Cryptographic RNG validation (FIPS 140 compliance).
- **Diehard Tests**
  - Developed by George Marsaglia
  - Year: 1995
  - Purpose: Early battery of tests for PRNGs; now considered somewhat outdated but still influential.
- **Dieharder**
  - Developed by Robert G. Brown (Duke University)
  - Year: 2003 (extended from Diehard)
  - Purpose: Modernized version of Diehard with additional tests.
- **TestU01**
  - Developed by Pierre L'Ecuyer & Richard Simard (University of Montreal)

  - Year: 2007
  - Purpose: Comprehensive testing with SmallCrush, Crush, and BigCrush batteries.

- **5. ENT (Entropy Assessment Tool)**
  - Developed by: John Walker (Fourmilab)
  - Year: 1998
  - Purpose: Quick entropy checks for random data.

- **PractRand**
  - Developed by: Melissa O'Neill (Harvey Mudd College)
  - Year: 2014
  - Purpose: Scalable testing for large datasets (up to TBs).

- **AIS-31**
  - Developed by: German Federal Office for Information Security (BSI)
  - Year: 2011
  - Purpose: Certification for hardware TRNGs in high-security applications.

- **FIPS 140-2/140-3 (Continuous RNG Tests)**
  - Developed by: NIST
  - Years: 2001 (FIPS 140-2), 2019 (FIPS 140-3)
  - Purpose: Mandatory for U.S. government cryptographic modules.

NIST SP 800-22 is the de facto standard for cryptographic validation. So, in the next section, we will discuss how the Test Suites work and the purpose of the tests

4.2 NIST Test for Random Bits

This document focuses on applications where randomness is required for cryptographic purposes. It describes a set of statistical tests for randomness, which help detect deviations in a binary sequence from randomness. The National Institute of Standards and Technology (NIST)[19] believes these procedures can help identify such deviations. However, testers should know that apparent deviations may stem from a poorly designed generator or anomalies within the binary sequence being tested. It is important to note

that a certain number of failures is expected in random sequences generated by a specific generator, and it is the tester's responsibility to interpret the test results correctly.

4.2.1 Testing

Hypothesis Testing for Randomness

In statistical hypothesis testing, the objective is to determine if a sequence of bits is random. This is done by establishing two competing hypotheses:

- **Null hypothesis (H_0)**: The sequence is random.
- **Alternative hypothesis (H_a)**: The sequence is non-random.

The statistical tests assess whether sufficient evidence exists to reject the null hypothesis, suggesting the sequence is non-random.

Test Statistic and Critical Value

A test statistic is calculated from the data based on the statistical test applied. Under the assumption that the sequence is random (H_0), this statistic follows a known reference distribution. A **critical value** is set, typically corresponding to a desired level of significance, which determines the threshold beyond which H_0 is rejected. If the test statistic exceeds this critical value, the sequence is considered non-random, and H_0 is rejected.

Error Types in Hypothesis Testing

Two types of errors can occur during hypothesis testing:

- **Type I error (α)**: The probability of rejecting H_0 when it is actually true (false positive).
- **Type II error (β)**: The probability of accepting H_0 when H_a is true (false negative).

In cryptographic applications, the significance level (α) is usually set at 0.01, meaning there is a 1% chance of incorrectly rejecting a truly random sequence.

P-value Interpretation

The **P-value** summarises the strength of evidence against the null hypothesis. It represents the probability that a random sequence would produce a sequence as non-random as the tested one. The decision rule is:

- If **P-value** $\geq \alpha$, accept H_0: The sequence appears random.

- If **P-value** ¡ α, reject H_0: The sequence appears non-random.

Significance Level (α)

The significance level (α) determines the threshold for rejecting the null hypothesis. Common values for α are:

- α = **0.001**: There is a 0.1% chance of incorrectly rejecting H_0 when it is true, suggesting 99.9% confidence that the sequence is random if H_0 is accepted.
- α = **0.01**: There is a 1% chance of rejecting H_0 when it is true, indicating 99% confidence in the randomness of the sequence.

For this context, α **is chosen to be 0.01**, meaning that if a sequence passes the test, there is a 99% confidence it is random.

4.2.2 Frequency (Monobit) Test

The focus of the test is the proportion of zeroes and ones for the entire sequence. This test determines whether a sequence's number of ones and zeros is approximately the same as expected for a truly random sequence. The test assesses the closeness of the fraction of ones to $\frac{1}{2}$; the number of ones and zeroes in a sequence should be about the same. All subsequent tests depend on passing this test.

4.2.3 Frequency Test within a Block

The focus of the test is the proportion of ones within M-bit blocks. This test aims to determine whether the frequency of ones in an M-bit block is approximately $M/2$, as expected under an assumption of randomness. For block size $M = 1$, this test degenerates to the Frequency (Monobit) test.

4.2.4 Runs Test

This test focuses on the total number of runs in the sequence, where a run is an uninterrupted sequence of identical bits. A run of length k consists of exactly k identical bits and is bounded before and after with a bit of the opposite value. The run test aims to determine whether a random sequence's number of runs of ones and zeros of various lengths is as expected. In particular, this test determines whether the oscillation between zeros and ones is too fast or too slow.

4.2.5 Test for the Longest Run of Ones in a Block

The focus of the test is the longest run of ones within M-bit blocks. This test aims to determine whether the longest run of ones within the tested sequence is consistent with the longest run of ones that would be expected in a random sequence. Note that an irregularity in the predicted length of the longest run of ones implies that there is also an irregularity in the predicted length of the longest run of zeroes. Therefore, only a test for ones is necessary.

4.2.6 Binary Matrix Rank Test

The focus of the test is the rank of disjoint sub-matrices of the entire sequence. This test aims to check for linear dependence among fixed-length substrings of the original sequence. Note that this test also appears in the DIEHARD battery of tests.

4.2.7 Discrete Fourier Transform (Spectral) Test

This test focuses on the sequence's peak heights in the Discrete Fourier Transform (DFT). This test aims to detect periodic features (i.e., repetitive patterns near each other) in the tested sequence that would indicate a deviation from the assumption of randomness. The intention is to detect whether the number of peaks exceeding the 95% threshold significantly differs from 5%.

4.2.8 Non-overlapping Template Matching Test

This test focuses on the number of occurrences of pre-specified target strings. This test aims to detect generators that produce too many occurrences of a given non-periodic (aperiodic) pattern. An m-bit window is used to search for a specific m-bit pattern for this test and the Overlapping Template Matching test. The window slides to a one-bit position if the pattern is not found. If the pattern is found, the window is reset to the bit after the found pattern and the search resumes.

4.2.9 Overlapping Template Matching Test

The Overlapping Template Matching Test aims to identify generators that produce too many occurrences of a specified pattern. Unlike the Non-overlapping Template Matching Test, if a pattern is found in this test, the window slides by only one bit before resuming the search.

4.2.10 Maurer's "Universal Statistical" Test

This test focuses on the number of bits between matching patterns (a measure related to the length of a compressed sequence). The test aims to detect whether or not the sequence can be significantly compressed without loss of information. A significantly compressible sequence is considered to be non-random.

4.2.11 Linear Complexity Test

This test focuses on a linear feedback shift register (LFSR) length. This test aims to determine whether or not the sequence is complex enough to be considered random. Longer LFSRs characterise random sequences. An LFSR that is too short implies non-randomness.

4.2.12 Serial Test

This test focuses on the frequency of all possible overlapping m-bit patterns across the sequence. This test determines whether the number of occurrences of the 2^m m-bit overlapping patterns is approximately the same as expected for a random sequence. Random sequences have uniformity; every m-bit pattern has the same chance of appearing as every other m-bit pattern. Note that for $m = 1$, the Serial test is equivalent to the Frequency test.

4.2.13 Approximate Entropy Test

As with the Serial test of Section 4.2.13, this test focuses on the frequency of all possible overlapping m-bit patterns across the entire sequence. The test compares the frequency of overlapping blocks of two consecutive/adjacent lengths (m and m+1) against the expected result for a random sequence.

4.2.14 Cumulative Sums (Cusum) Test

This test focuses on the random walk's maximal excursion (from zero) defined by the cumulative sum of adjusted (-1, +1) digits in the sequence. The test aims to determine whether the cumulative sum of the partial sequences occurring in the tested sequence is too large or too small relative to the expected behaviour of that cumulative sum for random sequences. This cumulative sum may be considered as a random walk. For a random sequence, the random walk excursions should be near zero. For certain types of non-random sequences, the excursions of this random walk from zero will be significant.

4.2.15 Random Excursions Test

This test focuses on the number of cycles having exactly K visits in a cumulative sum random walk. The cumulative sum random walk is derived from partial sums after the $(0, 1)$ sequence is transferred to the appropriate $(-1, +1)$ sequence. A cycle of a random walk consists of a sequence of steps of unit length taken at random that begin at and return to the origin. This test determines if the number of visits to a particular state within a cycle deviates from what one would expect for a random sequence. This test is a series of eight tests (and conclusions), one test and conclusion for each state: $-4, -3, -2, -1$ and $+1, +2, +3, +4$.

4.2.16 Random Excursions Variant Test

This test focuses on the number of times a particular state is visited (i.e., occurs) in a cumulative sum random walk. This test aims to detect deviations from the expected number of visits to various states in the random walk. This test consists of eighteen tests, one for each state: $-9, -8, \ldots, -1$ and $+1, +2, \ldots, +9$.

Chapter 5

Conclusion

5.1 Summary

A True Random Number Generator (TRNG) uses physical phenomena such as electronic noise, radioactive decay, or quantum events to generate truly random numbers, as contrasted with deterministic algorithms employed in Pseudo-Random Number Generators (PRNGs). A TRNG's product must be unpredictable, unbiased, and have high entropy to qualify for stringent uses such as cryptography, secure communication, and gaming. Raw TRNG outputs might occasionally be biased or non-uniform because physical processes have intrinsic imperfections.

Designing the TRNG requires three blocks: Entropy source, Digitisation and Post-processing. The entropy source is the primary building block of TRNG, which generates randomness from various noise sources and converts it into digital form by digitisation.

Raw TRNG outputs may still exhibit imperfections, so post-processing is often necessary to enhance their quality. Post-processing techniques remove biases, amplify entropy, and ensure that randomness suits high-security environments. Standard methods include whitening, which removes biases using XOR operations or linear transformations; compression, which reduces patterns to amplify entropy; and cryptographic functions, such as DES or block ciphers (e.g., AES), which generate uniformly random outputs. These operations convert raw TRNG data into high-quality random numbers, guaranteeing they satisfy the requirements of contemporary secure applications while being efficient and resilient. To overcome the disadvantage of cryptographic functions complexity developing the new method of post-processing using a Barrel shifter and XOR operation in that implementation, saw the challenges faced

To guarantee reliability, the NIST (National Institute of Standards and Technology) has implemented a set of statistical tests specified in Special Publication 800-22 to test the randomness and quality of TRNG outputs. They test different facets, including uniformity of bit distribution (Frequency Test), lack of patterns or streaks (Runs Test), randomness in longer bit blocks (Block Frequency Test), and total entropy levels (Approximate Entropy Test). Other tests are Cumulative Sums, Serial, and Longest Runs of Ones in a Block. All these tests mean that the numbers generated by the TRNG pass rigorous cryptographic and practical usage criteria.

5.2 Results

As a result, we implemented post-processing techniques using DES, AES, Barrel Shifter Only, Barrel Shifter + XOR (Final), Barrel Shifter + XOR (Neighbour Blocks) and Barrel Shifter + XOR (4-bit Focus). These methods reduced the autocorrelation between bits, as shown in the figures, and minimised the bias in the random bits.

5.2.1 NIST Test Results

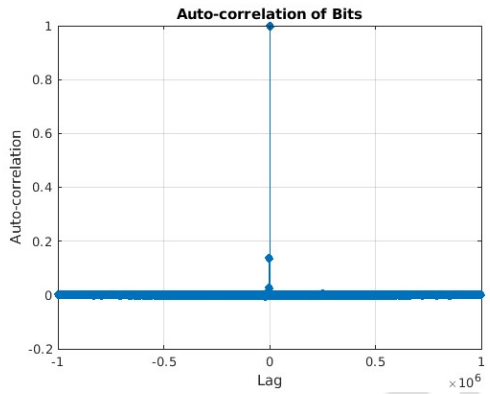
NIST Test	AES-128	DES	PRESENT-80	Barrel Shifter Only	Barrel Shifter + XOR(Final)	Barrel Shifter + XOR(Neighbor Block)	Barrel Shifter + XOR (4-bit Focus)
Frequency (Monobit)	PASS	PASS	PASS	FAIL	PASS	PASS	PASS
Block Frequency	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Runs	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Longest Run of Ones	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Rank	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Discrete Fourier Transform (FFT)	PASS	PASS	PASS	FAIL	PASS	PASS	PASS
Non-Overlapping Template Matching	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Overlapping Template Matching	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Maurer's Universal Statistical Test	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Linear Complexity Test	PASS	PASS	PASS	FAIL	PASS	PASS	PASS
Serial Test	PASS	PASS	PASS	FAIL	PASS	PASS	PASS
Approximate Entropy Test	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Cumulative Sums (Cusum) Test	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Random Excursions Test	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS
Random Excursions Variant Test	PASS	PASS	PASS	FAIL	FAIL	PASS	PASS

Table 5.1: NIST Test Results for PRESENT-80, AES-128, DES, Barrel Shifter, and Variations of Barrel Shifter + XOR

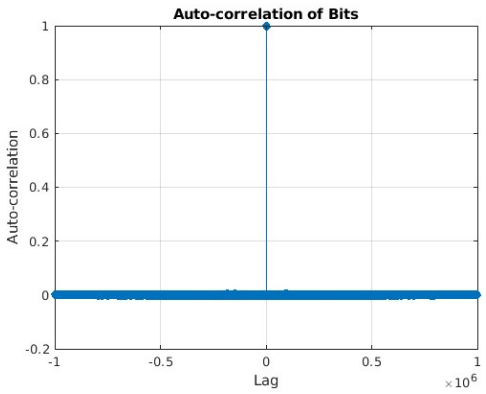
After testing several sequences of one million bits to the NIST test, where the DES, AES, PRESENT, and the Barrel Shifter + XOR (4-bit Focus) pass all the tests, but where Barrel Shifter + XOR(Neighbour Block) for some bit sequences pass all tests sometimes, it fails. In contrast, the Barrel Shifter + XOR(Final) fails to pass all the tests, and faintly, Barrel Shifter(Only) is the only one where it passes whatever input bit sequences and sometimes plus one. For Random Excursions Test and Random Excursions Variant Test for some sequences, it gets an error in all the algorithms of post-processing, like out-of-10 in a 2 or 3

5.2.2 Auto correlation Results

DES



(a) Auto correlation of input bits before post-processing



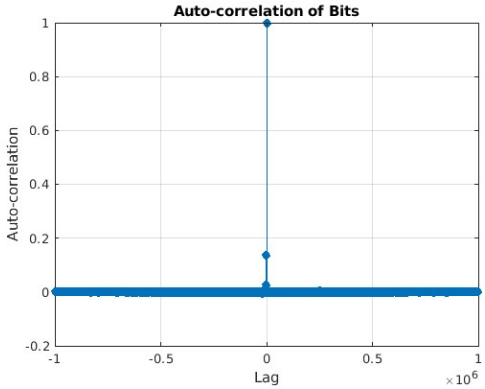
(b) Auto correlation after DES post-processing

Figure 5.1: Auto correlation of DES post-processing

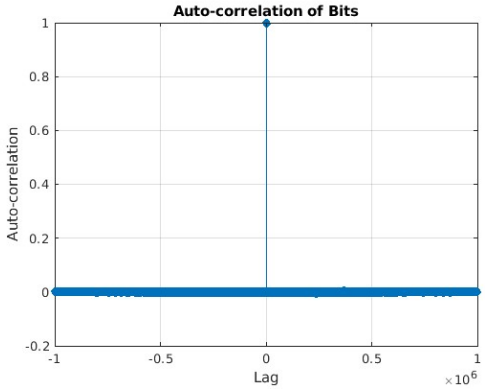
The DES code is implemented in MATLAB with a key size of 64 bits and 16 rounds, and the results were taken from MATLAB. From the graph observation, the autocorrelation was reduced from the input after the post-processing. It also passed all NIST tests, but due to our algorithm data, it is unsuitable for cryptography applications.

PRESENT

The PRESENT chipper goes 30 rounds and takes 80 bits of key. The code is implemented in MATLAB, and the results were obtained from MATLAB. From the graph observation, the autocorrelation was reduced from the input after the post-processing. It also passed all NIST tests, but it takes time due to more rounds.



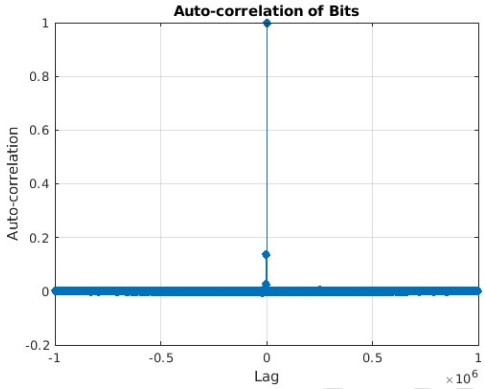
(a) Auto correlation of input bits before post-processing



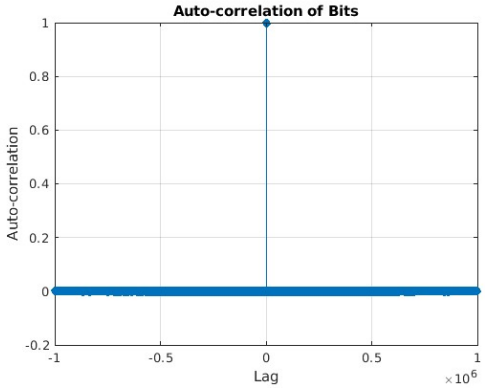
(b) Auto correlation after PRESENT post-processing

Figure 5.2: Auto correlation of PRESENT post-processing

AES



(a) Auto correlation of input bits before post-processing



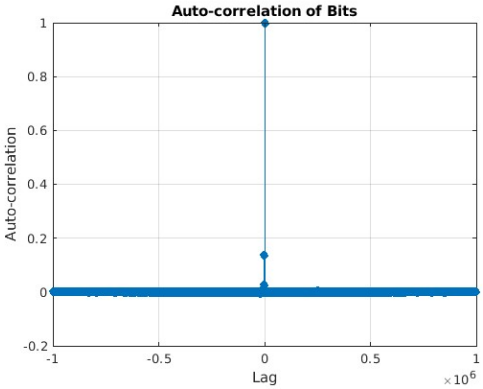
(b) Auto correlation after AES post-processing

Figure 5.3: Auto correlation of AES post-processing

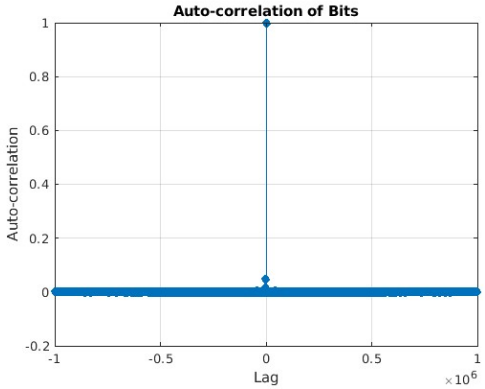
The results were taken from MATLAB and implemented in MATLAB with a key size of 128 bits and 10 rounds. From the graph observation, the autocorrelation was reduced from the input after the post-processing. It also passed all NIST tests, but due to being more complex and challenging to implement in Hardware like an FPGA, it required more area.

Barrel Shifter Only

Barrel Shifter Only is a simple and basic implementation of the proposed post-processing, which reduces the correlation marginally but not entirely like the DES and AES algorithms, and it takes hundreds of iterations to reduce the autocorrelation and fails to pass all NIST tests.



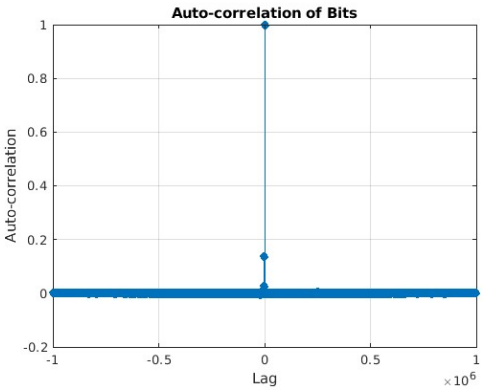
(a) Auto correlation of input bits before post-processing



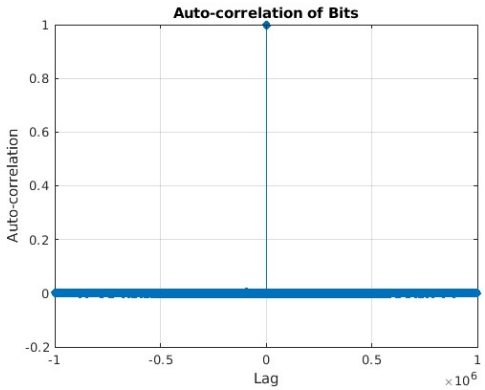
(b) Auto correlation after Barrel Shifter Only post-processing

Figure 5.4: Auto correlation of Barrel Shifter Only post-processing

Barrel Shifter + XOR (Final)



(a) Auto correlation of input bits before post-processing



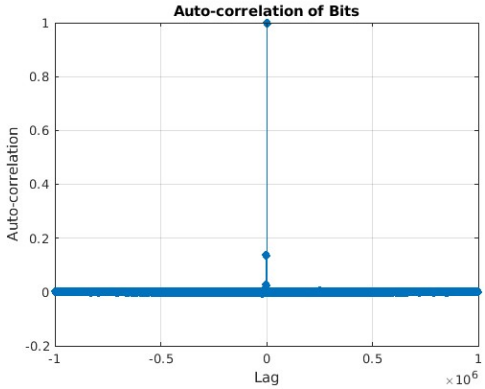
(b) Auto correlation after Barrel Shifter + XOR (Final) post-processing

Figure 5.5: Auto correlation of Barrel Shifter + XOR (Final) post-processing

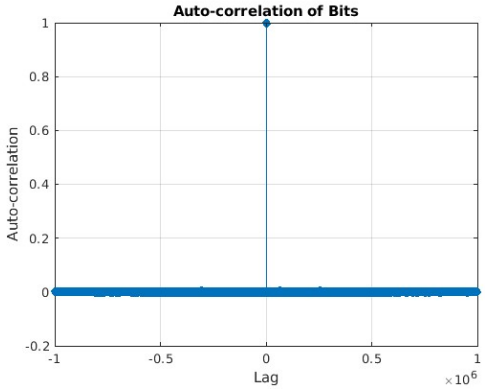
Barrel Shifter and XOR(final) reduce the autocorrelation and improve the randomness by reducing the bias in the bit sequence, but fail to pass all the NIST tests and take more iterations.

Barrel Shifter + XOR (Neighbor Blocks)

Barrel Shifter and XOR(Neighbour Blocks), where it reduces the autocorrelation and improves the randomness by reducing the bias in the bit sequence and passes all the NIST tests, but takes more iterations.



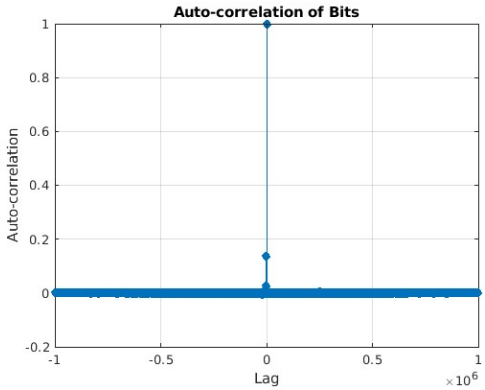
(a) Auto correlation of input bits before post-processing



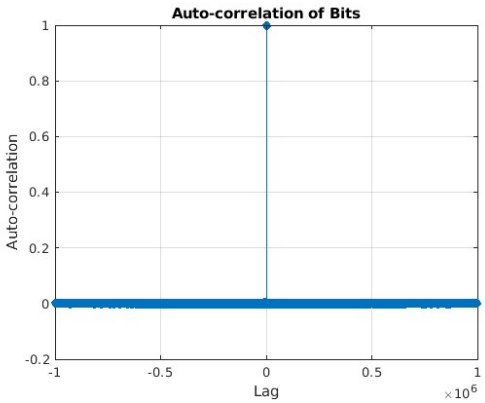
(b) Auto correlation after Barrel Shifter + XOR (Neighbor Blocks) post-processing

Figure 5.6: Auto correlation of Barrel Shifter + XOR (Neighbor Blocks) post-processing

Barrel Shifter + XOR (4-bit Focus)



(a) Auto correlation of input bits before post-processing



(b) Auto correlation after Barrel Shifter + XOR (4-bit Focus) post-processing

Figure 5.7: Auto correlation of Barrel Shifter + XOR (4-bit Focus) post-processing

Barrel Shifter and XOR (4-bit Focus) work as a cryptography algorithm to reduce autocorrelation and improve randomness by reducing the bias in bit sequences and passing all the NIST tests with fewer iterations.

5.3 Inferences

The applied post-processing methods—such as DES, PRESENT, AES, and Barrel Shifter with Shuffling—were very efficient in improving the statistical quality of the TRNG outputs. These methods effectively removed autocorrelation and minimised bias, allowing the processed random numbers to pass all 15 NIST statistical test suite tests. The experiments prove that the synergy of bit-level operations and cryptographic algorithms can be trusted to convert raw TRNG outputs into cryptographically secure random strings. Among the

tested methods, Barrel Shifter with XOR operations proved especially effective, providing strong bias correction with comparatively low computational overhead relative to traditional block cypher-based techniques. This systematic study shows that well-designed post-processing chains can successfully counter entropy constraints in TRNG outputs without compromising tight randomness requirements for security-critical uses. The results set a realistic paradigm for enhancing TRNG reliability via multi-stage processing, weighing stringent statistical correction against implementation practicality.

5.4 Conclusion

This research proves that the designed postprocessing pipeline, utilising Barrel Shifter with XOR operations, DES, and the PRESENT cypher, efficiently converts the raw TRNG output into cryptographically secure sequences. Most importantly, processed data satisfy all 15 NIST statistical tests, verifying the removal of detectable biases, autocorrelation, and entropy deficits. • entropy deficits. The Barrel Shifter approach is efficient, being NIST compliant with less computational overhead than standard block ciphers. These findings provide a reproducible paradigm for generating certifiable random numbers, satisfying high-security demands for use in applications, including key generation and cryptographic protocols.

5.5 Future work

Future work could systematically compare post-processing architectures, quantitative analysis of bias reduction and entropy preservation, and demonstrate a hardware-efficient solution combining bit-level shuffling with non-linear mixing. Adaptive parameter tuning is used to optimise throughput further while maintaining this rigorous standard of randomness.

Bibliography

[1] L. Gong, J. Zhang, H. Liu, L. Sang, and Y. Wang, "True random number generators using electrical noise," *IEEE Access*, vol. 7, pp. 125796–125805, 2019.

[2] X. Wu, Y. Ma, T. Chen, and N. Lv, "A distinguisher for rngs with lfsr post-processing," in *Information Security and Cryptology* (Y. Wu and M. Yung, eds.), (Cham), pp. 328–343, Springer International Publishing, 2021.

[3] C. Paar, *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer, 2010 ed., 2014.

[4] V. B. Suresh and W. P. Burleson, "Entropy and energy bounds for metastability based trng with lightweight post-processing," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 62, no. 7, pp. 1785–1793, 2015.

[5] S. Fujita, K. Uchida, S. Yasuda, R. Ohba, H. Nozaki, and T. Tanamoto, "Si nanodevices for random number generating circuits for cryptographic security," in *2004 IEEE International Solid-State Circuits Conference (IEEE Cat. No. 04CH37519)*, pp. 294–295, IEEE, 2004.

[6] M. Matsumoto, S. Yasuda, R. Ohba, K. Ikegami, T. Tanamoto, and S. Fujita, "1200μm 2 physical random-number generators based on sin mosfet for secure smart-card application," in *2008 IEEE International Solid-State Circuits Conference-Digest of Technical Papers*, pp. 414–624, IEEE, 2008.

[7] S. Yasuda, H. Satake, T. Tanamoto, R. Ohba, K. Uchida, and S. Fujita, "Physical random number generator based on mos structure after soft breakdown," *IEEE journal of solid-state circuits*, vol. 39, no. 8, pp. 1375–1377, 2004.

[8] S. Łoza and Ł. Matuszewski, "A true random number generator using ring oscillators and sha-256 as post-processing," in *2014 International Conference on Signals and Electronic Systems (ICSES)*, pp. 1–4, IEEE, 2014.

[9] B. Sunar, W. J. Martin, and D. R. Stinson, "A provably secure true random number generator with built-in tolerance to active attacks," *IEEE Transactions on computers*, vol. 56, no. 1, pp. 109–119, 2006.

[10] K. Wold and C. H. Tan, "Analysis and enhancement of random number generator in fpga based on oscillator rings," *International Journal of Reconfigurable Computing*, vol. 2009, pp. 1–8, 2009.

[11] C. Li, Q. Wang, J. Jiang, and N. Guan, "A metastability-based true random number generator on fpga," in *2017 IEEE 12th international conference on ASIC (ASICON)*, pp. 738–741, IEEE, 2017.

[12] V. B. Suresh and W. P. Burleson, "Entropy extraction in metastability-based trng," in *2010 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*, pp. 135–140, IEEE, 2010.

[13] A. Pandey and N. Kadayinti, "Trng based on multiple entropy sources using ctdsm," in *2024 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, pp. 534–538, 2024.

[14] S. T. Chandrasekaran, V. E. G. Karnam, and A. Sanyal, "0.36-mw, 52-mbps true random number generator based on a stochastic delta–sigma modulator," *IEEE Solid-State Circuits Letters*, vol. 3, pp. 190–193, 2020.

[15] S. T. Chandrasekaran, A. Jayaraj, N. Ramesh, and A. Sanyal, "33-200mbps, 3pj/bit true random number generator based on ct delta-sigma modulator," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–4, IEEE, 2021.

[16] K. K. e. Çetin Kaya Koç (auth.), *Cryptographic Engineering*. Springer US, 1 ed., 2009.

[17] A. M. Garipcan and E. Erdem, "Dessb-trng: A novel true random number generator using data encryption standard substitution box as post-processing," *Digital Signal Processing*, vol. 123, p. 103455, 2022.

[18] A. M. Garipcan and E. Erdem, "Design, fpga implementation and statistical analysis of a high-speed and low-area trng based on an aes s-box post-processing technique," *ISA Transactions*, vol. 117, pp. 160–171, 2021.

[19] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, A. Heckert, J. Dray, and S. Vo, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," Tech. Rep. SP 800-22 Rev. 1a, National Institute of Standards and Technology (NIST), 2010.